# Failover Strategies

*active-passive, active-active, and how warm the spare is*

Phase 3 videos on YouTube

**In one line:** active-passive keeps one serving site and a hot, warm or cold standby (simple data, slower and riskier failover); active-active serves from every site (no idle capacity, near-instant failover, but multi-site writes need conflict handling or partitioning); either way, detect carefully, avoid split brain, fail back slowly and rehearse.

## 1 Three in the morning

Your primary region goes down. Do you have a second region already serving traffic, one sitting idle waiting for this moment, or just backups and a runbook? The answer decides whether you're down for seconds, minutes or hours. Two numbers frame the choice (Phase 1, lesson 10): **RTO**, how long recovery may take, and **RPO**, how much recent data you may lose.

## 2 Active-passive and active-active

In **active-passive**, one site serves all traffic and a standby waits. When the primary fails, you **fail over**: the standby becomes primary. There's only one writer at a time, so data conflicts are rare. The costs: you pay for idle capacity, failover takes time, and it may not work if it's never tested.

In **active-active**, every site serves traffic all the time. If one fails, the others simply absorb its share. There's no idle capacity, and failover is nearly instant, because the other sites are already warm and proven by real traffic. The hard part is data: if several sites accept writes to the same record, you need conflict handling (Phase 2, database lesson 21), or you **partition** data so each record has one home site.

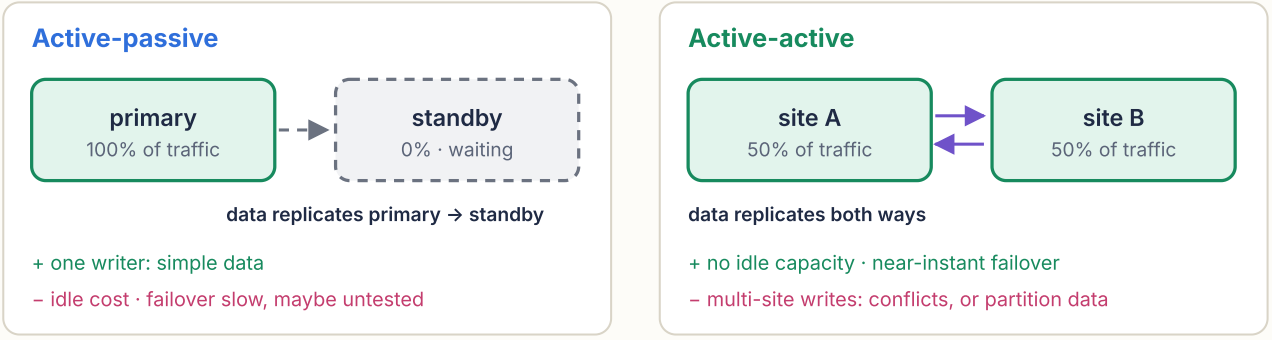



**Figure 1.** Active-passive keeps a spare; active-active uses every site. The difference shows up in cost, recovery time and data complexity.

## 3 How warm is the standby?

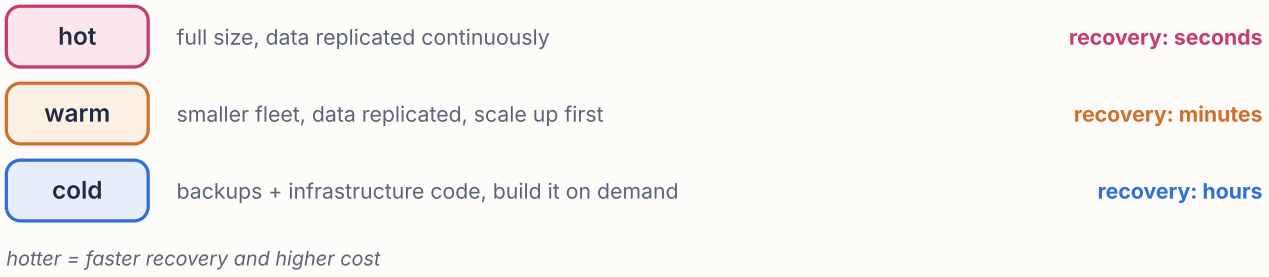



**Figure 2.** Standby temperatures: cheaper standbys recover more slowly.

Pick the temperature from the RTO: a payments system might need hot; an internal reporting tool can live with cold. Many teams use **warm**: the database replica is always running (that's what limits data loss), while the application fleet runs small and scales up during failover.

## 4 Detecting failure and moving traffic

**Detection**: health checks from several locations must agree before you act, so a network blip near one checker doesn't flip regions. **Moving traffic**: DNS failover changes the answer, but clients wait out the TTL; anycast withdraws a route and shifts within seconds (lesson 11).

Automate it where you can, but guard against **split brain**: two sites both believing they're primary and both accepting writes. The standard defence is **fencing**: before promoting the new primary, make sure the old one can no longer write (revoke its role, cut its access, or use a consensus system such as etcd or a cloud's managed failover to hold the leader lock).

**WORKED EXAMPLE: EAST US GOES DOWN**

Monitoring confirms the outage from three locations. The team, or automation, fences East US, promotes the database replica in **West US** to primary, scales the warm application tier up to full size, and switches traffic with Front Door or DNS. Writes resume in West US. Data written in East US after the last replication may be lost: with asynchronous replication lagging 5 seconds, the RPO is about 5 seconds.




**Figure 3.** A failover runbook for a warm standby. Fencing comes before promotion, to avoid two primaries.

## 5 Failback, and practice

When the old primary returns, don't rush back. **Resync** its data from the new primary (it may hold writes that never replicated: reconcile or discard them), verify it, and move traffic back gradually, or simply keep the new primary. And **practise**: run game days that fail over on purpose. A failover plan that's never been tested is a hope, not a plan.

**COMMON MISTAKES**

- A standby that was never tested, with expired credentials or a missing config, discovered during the outage.
- Promoting the replica without fencing the old primary: split brain and diverging data.
- Automatic failback that flaps between regions.
- Forgetting the other dependencies: DNS, secrets, queues and third-party allow-lists for the standby's IPs.

## 6 Key terms

**Failover**: moving service to a standby after a failure

**Failback**: returning to the original primary

**Active-passive**: one site serves, one waits

**Active-active**: every site serves all the time

**Hot / warm / cold**: how ready the standby is

**RTO / RPO**: time to recover / data you may lose

**Split brain**: two sites both acting as primary

**Fencing**: making sure the old primary can't write

## 7 Interview questions

**Q1. Compare active-active and active-passive setups.** BASIC

Active-active serves traffic from every site, so there's no idle capacity and failover is nearly instant, but multi-site writes need conflict handling or data partitioning. Active-passive keeps a hot, warm or cold standby, which is simpler for data but wastes capacity and makes failover slower and riskier.

**Q2. How do you fail back safely?** BASIC

Resync the old primary's data from the new primary and reconcile any unreplicated writes, verify it, shift traffic gradually (or keep the new primary), and rehearse failovers regularly.

**Q3. What decides hot, warm or cold standby?** BASIC

The RTO and RPO you need versus cost: hot for seconds of downtime, warm for minutes, cold for hours.

**Q4. Your RTO is 5 minutes and RPO 1 minute, on a budget. What setup?** SCENARIO

A warm standby: a continuously replicated database replica in the second region (RPO of seconds), a small application fleet that autoscales, infrastructure as code, and an automated, rehearsed runbook.

**Q5. What is split brain and how do you prevent it?** TRICKY

Two sites both believe they're primary and accept conflicting writes. Prevent it by fencing the old primary before promoting, using a quorum or a single lock holder (consensus) to decide who is primary, and requiring multiple observers to agree on failure.

**Q6. Why is active-active failover more trustworthy?** TRICKY

Every site already serves real traffic, so its capacity, configuration and dependencies are proven daily; a passive standby's problems only appear when you need it.

Next: SD-G-013 · Blue-Green and Weighted Traffic Splitting backendroadmap.com